



Facultad de Ingeniería Mecánica y Eléctrica

Universidad Autónoma de Nuevo León

Introducción al Aprendizaje Máquina

Grupo: 001 | Equipo: 01

Diseño y Entrenamiento de una Red Neuronal Artificial para Predicción de Accidente Cerebrovascular

Actividad Fundamental 2

Fecha: 13/11/2026

Profesor: Daniel Isais López Páez

Integrantes

Nombre	Matrícula	Hora clase
Uriel Samuel Bravo Castillo	2227015	V1 (LMV)
Vanessa Rodríguez Colunga	2158341	V1 (LMV)
Diego Alonso Carrillo Castillo	2144556	V1 (LMV)
Gustavo Guzmán Puente	2060982	V1 (LMV)

Índice

1. Objetivo **3**

2. Introducción **3**

3. Desarrollo **3**
3.1. Preprocesamiento Inicial y Codificación 3
3.2. Sección 1: División de datos y Escalado (Preparación para Deep Learning) . . . 4
3.2.1. Instrucciones y Procedimiento 4
3.2.2. Investigación: Necesidad del escalado en redes neuronales frente a árboles de decisión 4
3.3. Sección 2: Diseño de la Arquitectura y Selección de Funciones (Keras / TensorFlow) 5
3.3.1. Instrucciones y Arquitectura Implementada 5
3.3.2. Investigación y Diseño 6
3.4. Sección 3: Entrenamiento, Métricas y Optimizadores 6
3.4.1. Instrucciones y Configuración del Entrenamiento 6
3.4.2. Investigación: EarlyStopping y Comparación entre SGD y Adam 7

4. Resultados **8**

5. Conclusiones **9**

1. Objetivo

Diseñar, preprocesar, entrenar y evaluar una red neuronal artificial multicapa (Perceptrón Multicapa) mediante TensorFlow y Keras para la predicción binaria de accidentes cerebrovasculares (*stroke*) a partir de un conjunto de datos clínicos desbalanceados, aplicando técnicas de estratificación, estandarización de variables, regularización y detención temprana (*Early Stopping*) para optimizar métricas clínicas relevantes como el área bajo la curva ROC (AUC).

2. Introducción

Los accidentes cerebrovasculares (*stroke*) constituyen una de las principales causas de mortalidad y discapacidad permanente a nivel mundial. La identificación oportuna de pacientes con alto riesgo a partir de indicadores clínicos y demográficos —como la edad, hipertensión, cardiopatías, nivel promedio de glucosa e índice de masa corporal (BMI)— permite implementar intervenciones médicas preventivas de gran impacto.

En el ámbito del aprendizaje automático y profundo (*Deep Learning*), los modelos basados en redes neuronales artificiales ofrecen una alta capacidad para aproximar relaciones no lineales complejas entre variables clínicas heterogéneas [1]. Sin embargo, los conjuntos de datos médicos presentan de manera inherente un fuerte desbalance de clases, donde los casos positivos representan una fracción minoritaria de la población estudiada. Abordar este desafío exige una metodología rigurosa que abarque desde el preprocesamiento y escalado de características hasta la selección adecuada de funciones de activación, funciones de pérdida probabilísticas, optimizadores adaptativos y métricas de evaluación insensibles al sesgo por clase mayoritaria [2, 3].

3. Desarrollo

El código fuente completo, los cuadernos ejecutables de Google Colaboratory (`AF2.ipynb`) y el conjunto de datos utilizados en esta práctica se encuentran alojados y disponibles para su consulta en el repositorio oficial del equipo:

https://github.com/vanessardz-1203/IML

3.1. Preprocesamiento Inicial y Codificación

Previo a la construcción de la red neuronal, se cargó el archivo `healthcare-dataset-stroke-data.csv` compuesto por $5,110$ registros y 12 atributos originales. Se realizaron tres operaciones de limpieza:

1. **Imputación de valores faltantes:** Los valores nulos en la variable continua `bmi` fueron imputados utilizando la mediana de la distribución para evitar sesgos por valores atípicos.

2. **Eliminación de identificadores:** Se descartó la columna `id` al carecer de valor predictivo.
3. **Codificación One-Hot:** Mediante `pd.get_dummies(df, drop_first=True)`, las variables categóricas (`gender`, `ever_married`, `work_type`, `Residence_type` y `smoking_status`) fueron transformadas en variables binarias evitando la multicolinealidad perfecta, obteniendo un DataFrame final (`df_encoded`) de dimensiones $(5110, 17)$.

3.2. Sección 1: División de datos y Escalado (Preparación para Deep Learning)

3.2.1. Instrucciones y Procedimiento

Se separó el conjunto de datos preprocesado en la matriz de variables predictoras X (16 características) y el vector objetivo binario $y =$ `stroke`. Posteriormente, se aplicó una partición 80 %/20 % mediante `train_test_split` utilizando el parámetro `stratify=y` y `random_state=42`.

El parámetro `stratify=y` instruye a la función `train_test_split` para que divida los registros garantizando que la proporción de las clases (95.13 % negativos y 4.87 % positivos) se mantenga idéntica tanto en el conjunto de entrenamiento (4,088 muestras) como en el conjunto de prueba (1,022 muestras), replicando exactamente la distribución del dataset original. Finalmente, se ajustó un objeto `StandardScaler` exclusivamente sobre X_{train} (`fit_transform`) y se aplicó dicha transformación a X_{test} (`transform`) para evitar fuga de datos (*data leakage*) [4].

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df_encoded.drop(columns=['stroke'])
y = df_encoded['stroke']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Listing 1: División estratificada y estandarización de características

3.2.2. Investigación: Necesidad del escalado en redes neuronales frente a árboles de decisión

Pregunta: *¿Por qué es estrictamente necesario aplicar un escalador (como StandardScaler) o normalizar las variables continuas antes de ingresarlas a una red neuronal, a diferencia de*

otros algoritmos como los árboles de decisión?

Respuesta: La estandarización mediante `StandardScaler` transforma cada característica x_j para que tenga media cero ($\mu = 0$) y desviación estándar unitaria ($\sigma = 1$), según la ecuación:

$$z_j = \frac{x_j - \mu_j}{\sigma_j} \tag{1}$$

Su aplicación es estrictamente necesaria en redes neuronales artificiales por tres razones fundamentales [5, 1]:

- **Estabilidad y velocidad del Descenso de Gradiente:** Las redes neuronales ajustan sus pesos mediante algoritmos basados en gradientes. Cuando las variables poseen rangos muy dispares (por ejemplo, `age` entre 0.08 y 82, frente a `avg_glucose_level` entre 55.12 y 271.74), los pesos asociados a variables de mayor magnitud generan gradientes mucho mayores. Esto produce contornos elípticos muy elongados en la superficie de la función de costo, provocando oscilaciones y ralentizando severamente la convergencia.
- **Prevención de saturación en funciones de activación:** Entradas de gran magnitud empujan las combinaciones lineales $z = \mathbf{w}^T\mathbf{x} + b$ hacia las regiones planas de funciones como la sigmoide o tangente hiperbólica, donde sus derivadas tienden a cero (desvanecimiento del gradiente) y detienen el aprendizaje.
- **Invarianza en Árboles de Decisión:** A diferencia de las redes neuronales, los árboles de decisión realizan particiones ortogonales univariadas basadas en reglas de ordenamiento ($x_j \leq \theta$). Dado que cualquier transformación monótona creciente (como el escalado lineal) preserva el orden relativo de los valores y los puntos de corte que maximizan la ganancia de información o reducen la impureza de Gini, los árboles de decisión son completamente invariantes a la escala de los datos [6].

3.3. Sección 2: Diseño de la Arquitectura y Selección de Funciones (Keras / TensorFlow)

3.3.1. Instrucciones y Arquitectura Implementada

Se diseñó una arquitectura secuencial (`keras.Sequential`) compuesta por dos capas ocultas densamente conectadas (`Dense`) de 64 y 32 neuronas respectivamente, intercaladas con capas de regularización `Dropout` con tasa $p = 0.3$ para mitigar el sobreajuste, culminando en una capa de salida de una sola neurona. El modelo cuenta con un total de $3,201$ parámetros entrenables.

```
1 import tensorflow as tf
2 from tensorflow import keras
3 from tensorflow.keras import layers
4
5 model = keras.Sequential([
6     layers.Dense(64, activation='relu', input_shape=(X_train_scaled.shape
        [1],)),
7     layers.Dropout(0.3),
```

```
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])

model.summary()
```

Listing 2: Definición de la arquitectura secuencial en Keras

3.3.2. Investigación y Diseño

- **Función de activación en capas ocultas:** Se seleccionó la función **ReLU** (*Rectified Linear Unit*), definida como $f(z) = \max(0, z)$. Se eligió porque introduce no linealidad con un costo computacional mínimo, evita la saturación para valores positivos mitigando el problema del desvanecimiento del gradiente en redes profundas, y promueve representaciones dispersas (*sparsity*) eficientes [7].
- **Función de activación obligatoria en la capa de salida:** Para un problema de clasificación binaria (`stroke` $\in \{0, 1\}$), se debe emplear obligatoriamente la función logística **Sigmoide**:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

  Su rango de salida es el intervalo continuo $(0, 1)$, lo que permite interpretar directamente la activación de salida como la probabilidad posterior estimada $\hat{y} = P(y = 1 \mid \mathbf{x})$ [1].
- **Elección de `binary_crossentropy` frente a Error Cuadrático Medio (MSE):** En problemas de clasificación binaria con datos fuertemente desbalanceados, la entropía cruzada binaria (`binary_crossentropy`) se define como:

$$\mathcal{L}_{\text{BCE}}(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \tag{3}$$

  A diferencia del MSE, que asume ruido gaussiano y al combinarse con una salida sigmoide genera una superficie de error no convexa con gradientes muy pequeños cuando la predicción es extremadamente errónea ($\hat{y} \to 0$ cuando $y = 1$), la entropía cruzada derivada de la máxima verosimilitud de Bernoulli penaliza logarítmicamente los errores de confianza. Esto produce gradientes proporcionales al error $(\hat{y} - y)$, permitiendo que el modelo aprenda eficazmente sobre la clase minoritaria sin estancarse [2].

3.4. Sección 3: Entrenamiento, Métricas y Optimizadores

3.4.1. Instrucciones y Configuración del Entrenamiento

El modelo fue compilado utilizando el optimizador `adam`, la función de pérdida `binary_crossentropy` y un conjunto integral de métricas: `BinaryAccuracy`, `Precision`, `Recall` y `AUC`. Para el entrenamiento mediante `model.fit`, se reservó un 20 % de los datos de entrenamiento como

validación interna (`validation_split=0.2`), con un máximo de 100 épocas, tamaño de lote `batch_size=32` y el *callback* `EarlyStopping`.

```
from tensorflow.keras.callbacks import EarlyStopping

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=[
        tf.keras.metrics.BinaryAccuracy(name='accuracy'),
        tf.keras.metrics.Precision(name='precision'),
        tf.keras.metrics.Recall(name='recall'),
        tf.keras.metrics.AUC(name='auc')
    ]
)

early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True,
    verbose=1
)

history = model.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    callbacks=[early_stopping],
    verbose=1
)
```

Listing 3: Compilación y entrenamiento con EarlyStopping

3.4.2. Investigación: EarlyStopping y Comparación entre SGD y Adam

- **Funcionamiento de `EarlyStopping`:** Es un mecanismo de regularización temporal que supervisa una métrica de desempeño en el conjunto de validación (en este caso, `val_loss`) al finalizar cada época. Si dicha métrica no muestra una mejora durante un número consecutivo de épocas especificado por el hiperparámetro `patience` (10 épocas), el entrenamiento se detiene prematuramente para evitar que la red memorice el ruido del conjunto de entrenamiento (*overfitting*). Además, al activar `restore_best_weights=True`, el modelo revierte sus parámetros al estado exacto de la época donde se obtuvo el menor error de validación [2].
- **Diferencia fundamental entre SGD y Adam, y convergencia temprana:** El Descenso de Gradiente Estocástico (**SGD**) actualiza todos los parámetros del modelo utilizando una única tasa de aprendizaje global η fija (o con decaimiento programado) en

dirección opuesta al gradiente del mini-lote actual:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t) \tag{4}$$

Por su parte, **Adam** (*Adaptive Moment Estimation*) calcula tasas de aprendizaje adaptativas individuales para cada parámetro mediante el mantenimiento de promedios móviles exponenciales tanto del primer momento (la media del gradiente, equivalente a *Momentum*, m_t) como del segundo momento (la varianza no centrada del gradiente, similar a RMSprop, v_t), incorporando además una corrección de sesgo inicial [8]:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \tag{5}$$

Adam converge sustancialmente más rápido en las primeras épocas porque la corrección de sesgo ($\hat{m}_t, \hat{v}_t$) compensa la inicialización en cero de los momentos, mientras que el escalado inverso por $\sqrt{\hat{v}_t}$ amortigua las actualizaciones en direcciones con gradientes ruidosos o muy grandes y amplifica el paso en características poco frecuentes (como los casos positivos de *stroke*).

4. Resultados

Durante la ejecución del entrenamiento, el mecanismo `EarlyStopping` detuvo el proceso automáticamente en la **época 21**, identificando que el menor error de validación se alcanzó en la **época 11** y restaurando los pesos correspondientes a dicho punto óptimo.

Tabla 1: Métricas obtenidas en los hitos clave del entrenamiento y validación.

Hito del Entrenamiento	**Loss**	**Val Loss**	**AUC (Train)**	**Val AUC**
Época 1 (Inicialización)	0.3536	0.2143	0.5113	0.6352
Época 8 (Pico de Val AUC)	0.1602	0.1772	0.8140	0.8409
Época 11 (Mejor modelo restaurado)	**0.1553**	**0.1764**	**0.8317**	**0.8356**
Época 21 (Detención temprana)	0.1500	0.1798	0.8469	0.8301

Como se observa en la Tabla 1, el modelo logró una rápida reducción de la pérdida en las primeras 4 épocas gracias al optimizador Adam. Asimismo, se evidencia la importancia de monitorear la métrica **AUC** (0.8356 en validación en la época 11) frente a la exactitud bruta (`val_accuracy` = 0.9425): dado que el 95.1 % de las muestras pertenecen a la clase negativa, un clasificador trivial obtendría una exactitud superior al 94 % con sensibilidad nula, mientras que el valor de AUC demuestra que la red neuronal adquirió una sólida capacidad discriminativa para ordenar correctamente el riesgo relativo entre pacientes con y sin accidente cerebrovascular.

5. Conclusiones

- **Uriel Samuel Bravo Castillo:** La implementación de esta actividad permitió comprobar en la práctica cómo el preprocesamiento estratificado y la estandarización de variables continuas son indispensables para estabilizar el descenso de gradiente en redes neuronales. Asimismo, el análisis de los resultados demostró que, en problemas médicos con fuerte desbalance de clases, métricas como AUC-ROC resultan mucho más informativas que la exactitud convencional para evaluar la verdadera capacidad diagnóstica del modelo.
- **Vanessa Rodríguez Colunga:** El diseño de la arquitectura secuencial en Keras evidenció la sinergia entre las capas densas con activación ReLU, el uso de *Dropout* y la función de pérdida `binary_crossentropy`. La integración de `EarlyStopping` resultó clave para detener el entrenamiento en la época 21 y restaurar los pesos de la época 11, evitando el sobreajuste sobre los datos de entrenamiento y preservando la capacidad de generalización.
- **Diego Alonso Carrillo Castillo:** A través de la comparación teórica y experimental entre optimizadores y funciones de activación, se comprendió por qué Adam acelera drásticamente la convergencia en las primeras épocas mediante el ajuste adaptativo de tasas de aprendizaje por parámetro. De igual forma, el uso de `stratify=y` garantizó que tanto el conjunto de entrenamiento como el de prueba conservaran fielmente la proporción real del 4.87 % de casos positivos.
- **Gustavo Guzmán Puente:** Esta práctica consolidó el flujo completo de trabajo para problemas de clasificación binaria con aprendizaje profundo, desde la imputación de datos clínicos faltantes y codificación *One-Hot* hasta la compilación y validación del modelo. Se concluye que, aunque la red alcanza un excelente poder de discriminación (AUC > 0.83), en aplicaciones clínicas futuras sería recomendable complementar esta arquitectura con ajuste de umbral de decisión o ponderación de clases (`class_weight`) para incrementar directamente el *Recall*.

Anexos

A. Repositorio del Proyecto

Enlace clicable al repositorio oficial en GitHub (vanessardz-1203/IML)

B. Distribución de Clases Verificada en Ejecución

Durante la partición estratificada 80/20 del conjunto de datos (5,110 registros totales), se verificaron las siguientes proporciones exactas:

- **Conjunto de entrenamiento (X_{train}):** 4,088 muestras (95.1321 % clase 0, 4.8679 % clase 1).
- **Conjunto de prueba (X_{test}):** 1,022 muestras (95.1076 % clase 0, 4.8924 % clase 1).

Referencias

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: http://www.deeplearningbook.org

[2] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.

[3] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.

[4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] Y. A. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient backprop," in *Neural Networks: Tricks of the Trade*, 2nd ed., G. Montavon, G. B. Orr, and K.-R. Müller, Eds. Berlin, Heidelberg, Germany: Springer, 2012, pp. 9–48.

[6] L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen, *Classification and Regression Trees*. Boca Raton, FL, USA: CRC Press, 1984.

[7] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, Haifa, Israel, 2010, pp. 807–814.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

Las imágenes utilizadas en este documento son propiedad de sus respectivos autores y se incluyen únicamente con fines académicos.